# KNUTH-MORRIS PRATT ALGORITHM

ADVANCED ALGORITHMS

PAUL ROMER, KEVIN SHI, ALEX TALREJA

# MOTIVATION

Goal: Given a text and a pattern, find all occurrences of the full pattern within the text and return those indices

# NAÏVE APPROACH

Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match

| | | |
|---|---|---|
| A | C | F |

| | | | | | |
|---|---|---|---|---|---|
| A | C | D | A | C | F |

# NAÏVE APPROACH

Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match

| | | |
|---|---|---|
| A | C | F |

| | | | | | |
|---|---|---|---|---|---|
| A | C | D | A | C | F |

# NAÏVE APPROACH

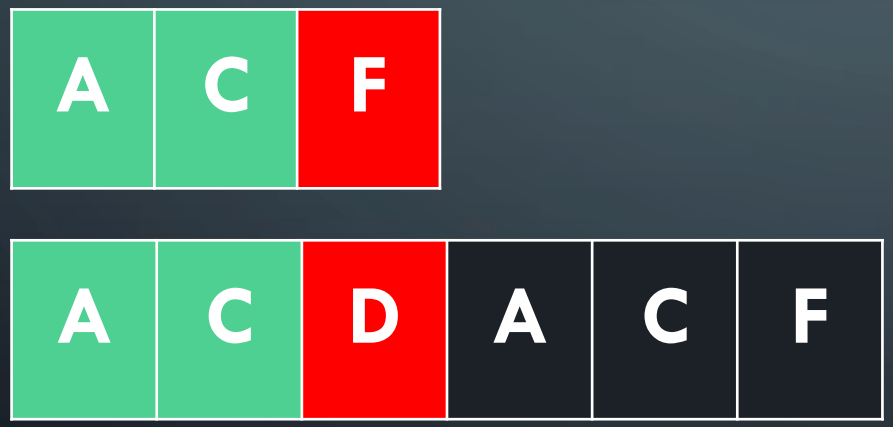
Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match

| | | |
|---|---|---|
| A | C | F |

| | | | | | |
|---|---|---|---|---|---|
| A | C | D | A | C | F |

# NAÏVE APPROACH

Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match



# NAÏVE APPROACH

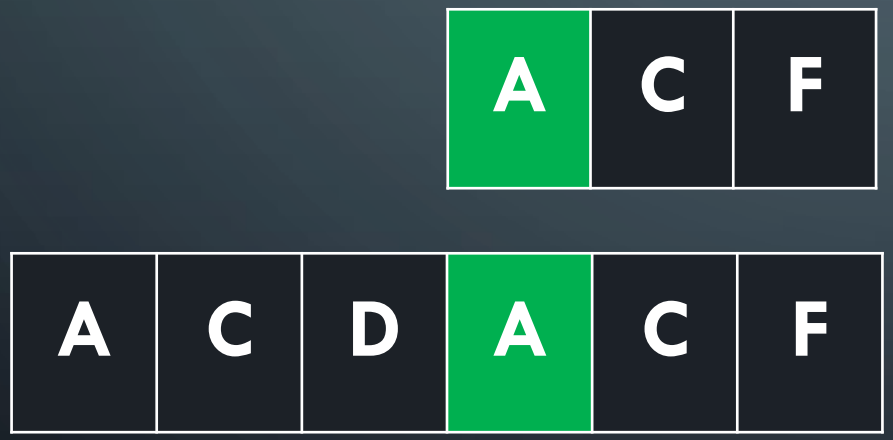
Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match

# NAÏVE APPROACH

Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match

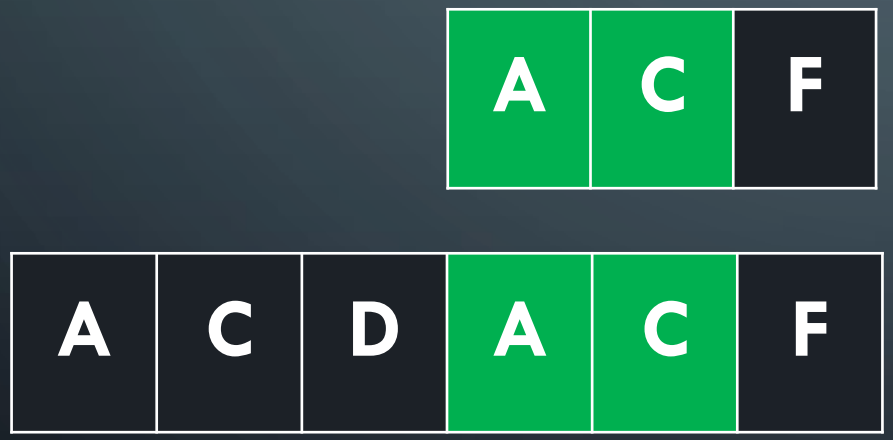
# NAÏVE APPROACH

Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match



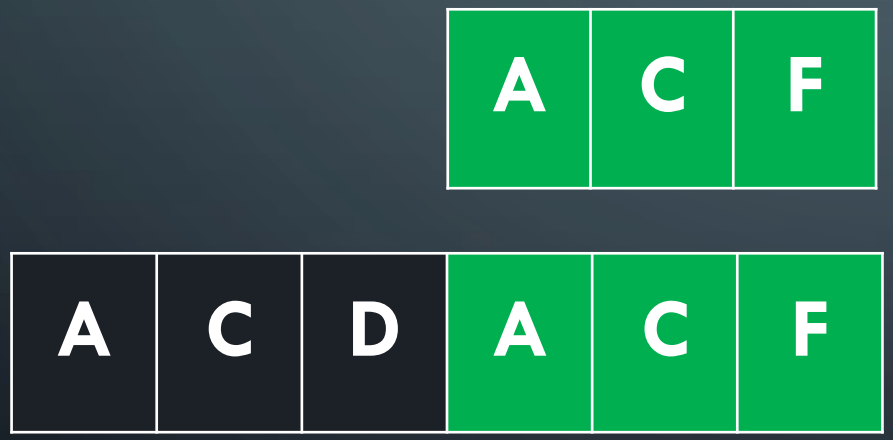
# NAÏVE APPROACH

Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match



# NAÏVE APPROACH

Given a text and pattern, at every index in the text see if using that as the start there is a full pattern match




Print/Store 3

# NAÏVE APPROACH RUNTIME

O(mn) for the naïve approach

M: length of the pattern

N: length of the text

| | | |
|---|---|---|
| A | C | F |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| B | C | A | C | D | A | C | F |

# PREFIX TABLE (IDEA/CONCEPT)

What if instead of losing all our progress at a mismatch, we had some way to know what the longest part of our current match could still be valid as a new starting point?

- Preventing us from fully restarting at a mismatch

We do this with a prefix table

- The prefix table tells us how many characters of the pattern we have already matched/seen

# BUILDING THE LPS TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | B | A | B | C |
| LPS | 0 | | | | |
| | J | i | | | |

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

# BUILDING THE LPS TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | B | A | B | C |
| LPS | 0 | 0 | | | |
| | J | i | | | |

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

# BUILDING THE LPS TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | B | A | B | C |
| LPS | 0 | 0 | 1 | | |
| | J | | i | | |

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

# BUILDING THE LPS TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | B | A | B | C |
| LPS | 0 | 0 | 1 | 2 | |
| | | J | | i | |

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

# BUILDING THE LPS TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | B | A | B | C |
| LPS | 0 | 0 | 1 | 2 | |
| | | | J | | i |

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

# BUILDING THE LPS TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | B | A | B | C |
| LPS | 0 | 0 | 1 | 2 | |
| | J | | | | i |

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

# BUILDING THE LPS TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | B | A | B | C |
| LPS | 0 | 0 | 1 | 2 | 0 |
| | J | | | | i |

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

# CORE IDEA: STOP MOVING BACKWARDS

Naïve Search:

Pointer jumps back to the start of the string after every mismatch. Time wasted.

KMP Search:

Text pointer **never** moves backwards. Use a Longest Prefix-Suffix (LPS) Table to reuse matches.

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | |
|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | | |
|---|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FIRST REAL WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 0 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| D | A | B | A | B | A | B | C |

Pattern

| | | | |
|---|---|---|---|
| A | B | A | B | C |

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# INTUITION

- m = length of pattern
- n = length of text

- Steps:
  - Prefix Table Construction: O(m)
  - KMP Search: O(n)

- Overall: O(n + m) = O(n)

# TIME COMPLEXITY

- m = length of pattern
- n = length of text

- Steps:
  - Prefix Table Construction: O(m)
  - KMP Search: O(n)

- Overall: O(n + m) = O(n)

```python
def LPS(pattern):
    m = len(pattern)
    lps = [0] * m
    j = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[j]
            j += 1
            lps[i] = j
            i += 1
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                lps[i] = 0
                i += 1

    return lps
```

```python
def search(pat, txt):
    n = len(txt)
    m = len(pat)
    lps = [0] * m
    res = []
    constructLps(pat, lps)
    i = 0
    j = 0
    while i < n:
        if txt[i] == pat[j]:
            i += 1
            j += 1
            if j == m:
                res.append(i - j)
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1]
            else:
                i += 1
    return res
```

# FALLBACK TABLE

- Key Optimization: skip fallback prefixes that are doomed
  - If a character match fails on 'A' at position 4, no part of the pattern can be used

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | A | A | A | A |
| LPS Table (standard) | 0 | 1 | 2 | 3 | 4 |

# FALLBACK TABLE

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | A | A | A | A |
| LPS Table (standard) | 0 | 1 | 2 | 3 | 4 |

T[i]: if chars don't match, how many chars can I keep (assumed matched)?

| | | | | | |
|---|---|---|---|---|---|
| -1 | -1 | -1 | -1 | -1 | 4 |

Fallback Table

T[i]: if chars don't match, where do I fallback to?

# WHY DO WE NEED THIS OPTIMIZATION?

- Consider the following text and pattern below:

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| Text | B | A | A | A | A | B | C | A |

| | | | | | |
|---|---|---|---|---|---|
| Pattern | A | A | A | A | A |

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

Not a match at index = 4

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

LPS is 4, go to pattern index = (4-1) = 3

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

Not a match at index = 4

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|

Text

| B | A | A | A | A | B | C | A |
|---|---|---|---|---|---|---|---|

Pattern

| A | A | A | A | A |
|---|---|---|---|---|

# WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| B | A | A | A | A | B | C | A |

Pattern

| | | | | |
|---|---|---|---|---|
| A | A | A | A | A |

# WALKTHROUGH

T =

| | | | | |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| B | A | A | A | A | B | C | A |

Pattern

| | | | | |
|---|---|---|---|---|
| A | A | A | A | A |

# IMPLEMENTATION DIFFERENCES

- Modify construction of prefix table

```python
def build_LPS_Table(pattern: str) -> list[int]:
    """
    Builds the standard (unoptimized) KMP fallback table.
    """
    m = len(pattern)
    table = [0] * (m + 1)

    pos = 1
    cnd = 0
    table[0] = -1

    while pos < m:
        table[pos] = cnd

        while (cnd >= 0) and (pattern[pos] != pattern[cnd]):
            cnd = table[cnd]
        pos += 1
        cnd += 1

    table[pos] = cnd
    return table
```

```python
if pattern[pos] == pattern[cnd]:
    table[pos] = table[cnd]
else:
    table[pos] = cnd
```

Store a compressed fallback if characters are the same (would require falling back further anyways)

# WALKTHROUGH

T =

| | | | | | |
|---|---|---|---|---|---|
| -1 | -1 | -1 | -1 | -1 | 4 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| B | A | A | A | A | B | C | A |

Pattern

| | | | | |
|---|---|---|---|---|
| A | A | A | A | A |

Not a match at index = 4

# WALKTHROUGH

T =

| | | | | | |
|---|---|---|---|---|---|
| -1 | -1 | -1 | -1 | -1 | 4 |

Need to shift entire pattern to -1 (beginning)

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| B | A | A | A | A | B | C | A |

Pattern

| | | | | |
|---|---|---|---|---|
| A | A | A | A | A |

Not a match at index = 4

# WALKTHROUGH

T =

| | | | | | |
|---|---|---|---|---|---|
| -1 | -1 | -1 | -1 | -1 | 4 |

Text

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| B | A | A | A | A | B | C | A |

Pattern

| | | | | |
|---|---|---|---|---|
| A | A | A | A | A |

# Q & A

ADVANCED ALGORITHMS

PAUL ROMER, KEVIN SHI, ALEX TALREJA